LogiEdge Phase 1 Report: Architecture, Hardware and Sensor Pipeline

AIML ZG535 Machine Learning on Edge (EC-I)

Group 15

Student Details:

BITS ID	Name	Contributions
2025ae05381	Disha Shetty H	100%
2025ae05382	Murali Krishna Sai Garisapati	100%
2025ae05420	Rajan Yadav	100%
2025ae05680	Santosh Kumar Behera	100%

Repository: https://github.com/santosh-2025ae05680/logibridge

1. Scope

This report covers Components A to C (Modules 1 to 3) of LogiEdge, the cold-chain monitoring platform for FreightBridge's 85 refrigerated trucks: the deployment justification and system architecture, hardware selection with the Roofline analysis, and the sensor simulator, MQTT design and preprocessing pipeline. All sensor data is simulated. Code, outputs and figures are in the repository. The mandatory 3σ normalisation experiment is reported in the Phase 2 report.

2. Deployment context and edge constraints (Task A1)

FreightBridge carries vaccines and medicines on routes where cellular coverage is unreliable, and has already lost ₹28 lakh of vaccine to a refrigeration failure that went undetected on the Nashik–Aurangabad route. The system must classify the cargo compartment as Normal, Warning or Critical on the truck and alert the operations centre without depending on continuous coverage. Table 1 summarises the four constraints that justify running inference at the edge.

Table 1: Edge AI constraints

Constraint	Numbers	Consequence
Latency	Fault raises temperature about 1 °C/min; a Critical breach (above 3 °C off setpoint) appears about 3 min after the fault; the SLA is 90 s; edge detection takes about 41 s in the worst case	On-device inference leaves about 49 s of margin

Constraint	Numbers	Consequence
Bandwidth	Raw streams are about 518.7 MB per truck per day (₹51.87 per day, about ₹1.32 lakh per month for 85 trucks); alerts alone are about 0.003 MB per day	Process on the truck and send alerts only, about 170,000 times less data
Connectivity	Seven locations lose coverage for 35 to 90 minutes	Classification and alert logging must continue offline, with store-and-forward
Privacy	Pharmaceutical clients need proof that cargo data is not exposed	Raw streams stay on the truck; the broker listens on 127.0.0.1; only alerts travel upstream over TLS

Latency. Worst-case detection is bounded by the pipeline design. A fault that starts just after a window closes must fill a 30-second window, and windows close every 10 seconds, so detection takes at most about 40 seconds, plus microseconds to milliseconds of inference and under a second to log and publish. Rural cellular round-trip time [CITE: figure and source] would also fit within 90 seconds while connected. The real problems are tail delays, retries and, above all, absent coverage.

Bandwidth. The figures assume 4 bytes per sample (float32), decimal megabytes, 40 door events a day of 50 bytes, 10 alerts a day of 300 bytes and ₹0.10 per MB. Vibration dominates: 500 Hz on three axes for 86,400 s is 518.4 MB a day, against 0.35 MB for temperature and 0.002 MB for door events. The alert volume is an assumption. A heartbeat every five minutes would raise it, but it would still be thousands of times smaller than raw streaming.

Connectivity. During a gap a cloud-only design can classify nothing, and a fault that starts mid-gap stays undetected well beyond the three-minute critical threshold. LogiEdge keeps sensing, the broker, inference and alert logging on the truck, queues alerts in a persistent local log and forwards them in order, with original timestamps, when coverage returns. This also gives a complete chain-of-custody record.

3. System architecture (Task A2)

[Figure 1: System architecture. Insert scenario_architecture/system_architecture.png]

Sensors (temperature probe at 1 Hz, a three-axis accelerometer on the compressor, a door switch) publish to a local Mosquitto broker on the edge node. A containerised service performs preprocessing and TFLite classification, publishes each result and writes alerts to a persistent local log. An uplink agent forwards alerts to the operations centre (alert ingestion, database, dashboard) over the intermittent cellular link. Everything inside the truck boundary runs with no connectivity; only the uplink depends on coverage. The uplink agent is part of the design only and is not implemented.

4. Hardware selection (Task B1)

Three options were compared against the 10 W AI power budget from the truck's 12 V supply (Table 2).

Table 2: Hardware options

Option	Hardware	Price per truck	Power	Within 10 W	Pilot (85)	Full (265)	12 V current	Energy per day
1	Raspberry Pi 5 (8 GB) + AI HAT+ (Hailo-8L)	₹15,000	7.5 W	Yes	₹12.75 lakh	₹39.75 lakh	0.69 A	180 Wh
2	Jetson Orin Nano Super	₹45,000	15 W	No (150%)	₹38.25 lakh	₹119.25 lakh	1.39 A	360 Wh
3	STM32H7 custom MCU	₹3,500	0.4 W	Yes	₹2.98 lakh	₹9.28 lakh	0.04 A	9.6 Wh

The 12 V current assumes 90% DC-DC converter efficiency.

Constraint Triangle. Option 1 is the balanced point of the triangle: ample performance on a full Linux platform, power inside the budget with 2.5 W of headroom, and mid-range cost. Option 2 sits at the extreme performance corner but breaks the power budget and carries the highest cost. Option 3 occupies the best power and cost corner but offers only a bare-metal or RTOS stack.

Dominant vertex: power. Power is the only hard pass/fail limit in the deployment. Latency does not discriminate between the options: the Roofline bound of about 2.8 ms is some 32,000 times inside the 90 s SLA for every option. Cost ranks the options that pass the power test without eliminating any, and cost alone would pick the STM32, which cannot host the required software stack. Power also drives operating risk through cab heat and continuous current draw.

Recommendation: Option 1. The Pi 5 meets the budget at 7.5 W, runs the architecture as designed (local Mosquitto broker, Docker container, Ansible deployment, registry-based updates) and costs one third of the Jetson. The Jetson exceeds the budget by 50% at three times the cost for compute a model this small cannot use (a lower-power mode may exist [verify in NVIDIA documentation], but it would throttle the performance the extra cost pays for). The STM32 is cheapest, but it cannot run Docker, Mosquitto or Ansible, on which container-based updates depend. The premium for the Pi is ₹9.78 lakh at pilot scale (₹30.48 lakh at 265 trucks) and buys a platform that can be updated every six weeks.

Caveats. The Hailo accelerator is not needed for this model, and it requires models compiled to its own format rather than plain TFLite [verify]. In-vehicle conditions (12 V transients, cab temperature, the Pi 5's rated ambient range [verify against the datasheet]) call for an automotive-grade DC-DC converter and enclosure. The selection is analysis only: all measurements in this project ran on a laptop.

5. Arithmetic intensity and Roofline (Task B2)

Table 3: Roofline analysis (Pi 5 CPU)

Quantity	Value
Arithmetic intensity (45 MFLOP / 18 MB)	2.50 FLOP/byte

Quantity	Value
Ridge point (16 GFLOP/s / 12 GB/s)	1.33 FLOP/byte
Classification	Compute-bound (intensity above the ridge point)
Attainable performance	16.00 GFLOP/s
Compute time / memory time	2.812 ms / 1.500 ms
Latency lower bound	2.812 ms (about 32,000 times inside the 90 s SLA)
Compute time with 35% fewer FLOPs	1.828 ms

The model lies right of the ridge point, on the flat compute roof, so latency is limited by arithmetic throughput. The Roofline therefore favours reducing arithmetic work or raising compute throughput: INT8 quantisation (integer SIMD, which also cuts memory traffic about four times), structured pruning, or a smaller network. Reducing memory traffic alone would not help, because memory time is already below compute time. The trained network is far smaller than the exercise figures (about 800 parameters, roughly 1.5 kFLOPs), so the real model has far more latency headroom.

6. Sensor simulator and MQTT design (Task C1)

A Python simulator publishes three streams to the local broker and accepts `--anomaly {none|temp_drift|vibration|combined}`. Further flags set the start and end of an anomaly (`--anomaly-after`, `--anomaly-until`), a time-acceleration factor with simulated timestamps (`--speed`), a random seed, and the mean door interval.

Table 4: Simulated streams

Stream	Rate	Normal	Anomaly
temperature	1 Hz	N(4.0 °C, 0.3)	temp_drift: +0.08 °C per reading
vibration_rms	0.5 Hz	N(0.45 g, 0.05)	vibration: step to N(1.2 g, 0.15)
door	on event	OPEN/CLOSE with timestamp (open 15 to 90 s, mean gap 300 s)	combined: both anomalies together

The drift is implemented literally as specified. It is 4.8 °C per minute, far steeper than the 1 °C per minute of a real failure, and ends about 72 °C above setpoint after 15 minutes. This is recorded as a limitation. Table 5 checks the generator against the specified parameters over 15 simulated minutes.

Table 5: Generator check (900 samples per mode)

Mode	Temp mean	Temp std	First	Last	Vibration mean	Vibration std
none	3.99	0.31	4.29	3.44	0.447	0.049
vibration	3.99	0.31	4.29	3.44	1.191	0.147

Mode	Temp mean	Temp std	First	Last	Vibration mean	Vibration std
temp_drift	40.03	20.79	4.37	75.44	0.447	0.049
combined	40.03	20.79	4.37	75.44	1.191	0.147

Topic tree and QoS. [Figure 2: MQTT topic tree. Insert the rendered diagram from data_pipeline/mqtt_architecture.md] All topics sit under `logibridge/trucks/{truck_id}/`. The truck ID is in the topic, so sensor payloads carry only a timestamp and a value; alerts carry the truck ID because they leave the truck. Payloads are compact JSON.

Table 6: Topic design

Topic	Publisher to subscriber	Rate	QoS	Retained
sensors/temperature	simulator to inference service	1 Hz	0	no
sensors/vibration_rms	simulator to inference service	0.5 Hz	0	no
sensors/door	simulator to inference service	on event	1	no
inference	inference service to drift monitor	every 10 s	0	no
alerts	inference service to alert log and uplink agent	on Warning or Critical	1	no
status/{client}	each client (simulator, inference service)	on change	1	yes, with Last Will

- **Temperature and vibration (QoS 0):** high-rate streams that the next reading supersedes within 1 to 2 s. Delivery is over the loopback interface, and QoS 1 would add an acknowledgement per message and an in-flight queue that can grow if the broker stalls. A lost sample is harmless because of the moving average over 30 s windows.
- **Door (QoS 1):** rare events whose loss would corrupt the door-state history and the audit trail. At-least-once is enough, because a duplicate event with the same timestamp is harmless.
- **Inference (QoS 0):** replaced every 10 s, and the drift monitor works on the distribution of the last 100 results, so it tolerates an occasional loss.
- **Alerts (QoS 1):** the safety-critical messages must arrive at least once, including after an outage. QoS 2 is not used because its four-message handshake adds round trips on a high-latency rural link, and a unique alert ID lets the operations centre discard duplicates more cheaply.
- **Status (QoS 1, retained, Last Will):** a late subscriber immediately sees whether a client is online, and the broker publishes offline if a client dies without disconnecting.

The delivered QoS is the lower of the publisher's and subscriber's requested QoS, so subscribers to door events and alerts must subscribe with QoS 1. The broker listens only on 127.0.0.1. The uplink agent uses a persistent session with QoS 1, so queued alerts survive coverage gaps.

7. Preprocessing pipeline (Task C2)

Each stream passes a causal 5-sample moving average, because a live stream cannot look ahead. Windows of 30 s with a 10 s step then produce a six-value feature vector: temperature mean, temperature standard deviation, temperature rate of change (least-squares slope, °C per minute), vibration RMS, vibration peak and vibration kurtosis. A window closes when the first later sample arrives, so every window is complete (about 1 s extra latency), and the first window ends at 38 s so that no moving-average warm-up values enter it. Features are z-scored with a mean and standard deviation computed once from 10 minutes of clean Normal data, saved as `training_stats.npy` and never recomputed from live data, and then clipped to ±10.

Table 7: Mean raw features per mode (87 windows each)

Mode	Temp mean	Temp std	Temp rate (°C/min)	Vib RMS	Vib peak	Vib kurtosis
none	3.987	0.127	0.001	0.448	0.481	-0.594
temp_drift	40.187	0.715	4.801	0.448	0.481	-0.594
vibration	3.987	0.127	0.001	1.193	1.292	-0.594
combined	40.187	0.715	4.801	1.193	1.292	-0.594

Table 8: Normalisation statistics (57 clean windows, seed 42)

Feature	Mean	Std
temp_mean	3.9922	0.0442
temp_std	0.1189	0.0181
temp_rate_c_per_min	0.0066	0.2635
vib_rms	0.4510	0.0126
vib_peak	0.4836	0.0185
vib_kurtosis	-0.6517	0.8437

Normal windows then score between -0.26 and 0.43 on every feature, while anomalous windows are pinned at +10. **Why clip.** Drift windows produce z-scores in the hundreds, which would stretch the INT8 input range and crush the resolution available to Normal data; Phase 2 shows that the unclipped FP32 model also fails.

Kurtosis carries no class information. It is -0.594 in every mode, because a shift and scale of Gaussian noise (0.45 + 0.05z becomes 1.2 + 0.15z) leaves kurtosis unchanged, and a 15-sample estimate is noisy and biased low. It would matter for real bearing wear, which produces impulsive, heavy-tailed signals. The feature is kept because the six-value vector is specified, and the model learns to ignore it.

8. Data fusion (Task C3)

LogiEdge uses feature-level fusion: features are extracted from each sensor independently and concatenated into one six-value vector before the model.

- **Data-level fusion** would merge raw streams that differ in rate (1 Hz and 0.5 Hz), units and noise, and would need a larger model on a 10 W node.
- **Decision-level fusion** would run a model per sensor and combine verdicts. Warning and Critical differ by the combination of temperature and vibration, so per-sensor models cannot see the interaction, and the design needs extra models and a combination rule.
- **Feature-level fusion** keeps the model small, preserves the cross-sensor interaction, and lets one set of normalisation statistics cover the whole vector.

9. Evidence of operation

[Attach screenshots or output excerpts: (a) `mosquitto_sub -t "logibridge/#" -v` while the simulator runs in each mode, showing the sensor, door and status topics; (b) `python data_pipeline/preprocessing.py check` and `compute-stats`; (c) `python data_pipeline/preprocessing.py live` printing raw and normalised windows, with z-scores near 0 until an anomaly starts and then clipped at +10.]

10. Assumptions, limitations and next steps

- Bandwidth assumes float32 samples, 40 door events and 10 alerts per day; the raw 500 Hz vibration stream is used only in the bandwidth calculation, and the simulator publishes the 0.5 Hz RMS stream as specified.
- The literal drift (4.8 °C per minute) is unrealistically steep; an optional drift cap exists but is not used.
- Kurtosis is uninformative on Gaussian fault data; the hardware selection is analysis only; the uplink agent is not implemented.
- Phase 2 covers the dataset, model, TFLite conversion, the 3σ experiment, Docker deployment, drift monitoring and Ansible.